UNISENAI
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (EAD)

VICTOR GABRIEL JAEGER
VINICIUS DE ASSIS MENDES
VINÍCIUS ITAMAR DE SOUZA

DESAFIO 2 - MODELAGEM E ESTRUTURAÇÃO DO BANCO DE DADOS

FLORIANÓPOLIS
2026

VICTOR GABRIEL JAEGER
VINICIUS DE ASSIS MENDES
VINÍCIUS ITAMAR DE SOUZA

DESAFIO 2 - MODELAGEM E ESTRUTURAÇÃO DO BANCO DE DADOS

Trabalho apresentado ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas (EAD) do UNISENAI Campus Florianópolis, como requisito parcial de avaliação da disciplina Projeto Aplicado I .

Professor: Hugo Menezes Barra

FLORIANÓPOLIS

2026

1. Objetivo

O presente trabalho tem como objetivo a modelagem e a implementação inicial de um banco de dados relacional destinado à organização das informações utilizadas no processo de calibração de equipamentos.

Na primeira entrega, foram identificadas necessidades relacionadas à organização dos dados provenientes de diferentes planilhas eletrônicas, especialmente quanto à identificação dos equipamentos, ao registro das medições e à rastreabilidade das informações necessárias à elaboração dos certificados de calibração.

Dando continuidade ao levantamento realizado, a segunda entrega contempla a estruturação de um banco de dados SQLite, organizado em três tabelas principais: técnico, equipamento e medição. Essa divisão busca reduzir a repetição desnecessária de informações cadastrais e estabelecer relacionamentos entre os registros por meio de chaves primárias e estrangeiras.

A implementação foi desenvolvida no Visual Studio Code, utilizando comandos SQL para a definição das tabelas e scripts Python para sua execução e verificação. Foram realizados testes de inserção, consulta e integridade referencial, com a finalidade de verificar o funcionamento básico da estrutura criada.

Ressalta-se que esta implementação corresponde a um modelo inicial e simplificado de armazenamento de dados, não abrangendo, nesta etapa, todas as funcionalidades de processamento metrológico, validação técnica e emissão automática de certificados previstas no levantamento de requisitos.

2. Organização e Modelagem dos Dados

O banco de dados foi estruturado em três tabelas principais: tecnico, equipamento e medicao. Essa organização permite separar as informações cadastrais dos registros de medição, evitando que os mesmos dados precisem ser repetidos a cada nova operação.

A tabela tecnico armazena as informações dos profissionais responsáveis pelas medições. Os campos definidos são o identificador do técnico, nome, matrícula, e-mail e setor de atuação. Cada profissional recebe um identificador único, permitindo seu relacionamento com diferentes medições.

A tabela equipamento reúne os dados dos instrumentos submetidos à medição ou calibração. Sua estrutura contém o identificador do equipamento, código de identificação, nome, fabricante e modelo. O cadastro individual permite que um mesmo equipamento esteja relacionado a diferentes registros de medição sem a necessidade de repetir suas informações.

A tabela medicao registra os resultados obtidos durante a execução das medições. Entre os campos definidos estão o parâmetro medido, valor nominal, tolerâncias mínima e máxima, valor obtido, status do resultado e data do registro. Essa tabela também contém os identificadores do técnico responsável e do equipamento correspondente.

A separação dessas informações contribui para uma estrutura mais organizada, facilita a consulta dos registros e reduz a repetição desnecessária dos dados cadastrais.

3. Relacionamento Entre As Tabelas

O relacionamento entre as tabelas foi estabelecido por meio de chaves primárias e estrangeiras.

As chaves primárias (Primary Keys - PK) são responsáveis pela identificação única de cada registro dentro de uma tabela. No projeto, foram utilizadas as chaves id_tecnico, id_equipamento e id_medicao.

As chaves estrangeiras (Foreign Keys - FK) permitem estabelecer relações entre tabelas diferentes. Na tabela medicao, o campo id_tecnico faz referência à tabela tecnico, enquanto o campo id_equipamento faz referência à tabela equipamento.

Dessa maneira, cada medição deve estar vinculada a um técnico e a um equipamento previamente cadastrados. Ao mesmo tempo, um técnico pode estar associado a diferentes medições, assim como um equipamento pode possuir diversos registros.

Para reforçar a integridade dos relacionamentos, foi utilizado o comando PRAGMA foreign_keys = ON, responsável por ativar a verificação das chaves estrangeiras na conexão SQLite.

Também foram definidas restrições para impedir a exclusão de técnicos ou equipamentos que possuam medições vinculadas, contribuindo para a preservação das referências entre os registros.

4. Dicionário e Tipos de Dados

O dicionário de dados foi desenvolvido no arquivo 02_dicionario.md, contendo a identificação dos campos utilizados nas três tabelas do sistema.

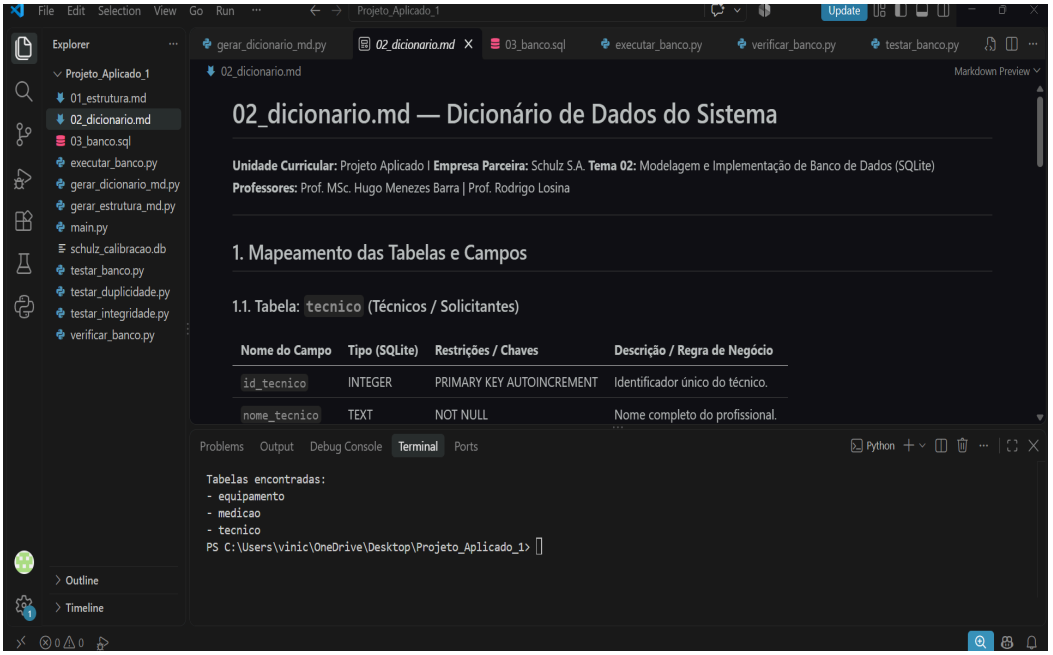
Foram documentados 20 campos, distribuídos entre as entidades tecnico, equipamento e medicao. Para cada campo, foram definidos seu nome, tipo de dado, restrições aplicáveis e finalidade dentro da estrutura.

Os principais tipos utilizados foram:

- INTEGER: utilizado para armazenar números inteiros, especialmente os identificadores dos registros.
- TEXT: utilizado para armazenar informações textuais, como nomes, matrículas, códigos e descrições.
- REAL: utilizado para armazenar valores numéricos com casas decimais, como valores nominais, tolerâncias e resultados de medição.
- DATETIME: declaração utilizada para o campo de data e hora da medição, com preenchimento padrão por meio de CURRENT_TIMESTAMP.

O banco também utiliza restrições como NOT NULL, para impedir valores nulos em campos obrigatórios, e UNIQUE, para evitar a repetição de informações que devem ser exclusivas, como a matrícula do técnico e o código do equipamento.

O dicionário foi gerado por meio do script Python gerar_dicionario_md.py, permitindo organizar a documentação dos campos em formato Markdown.



Fonte: Elaborado pelos autores (2026).

5. Testes e Verificação do Banco de Dados

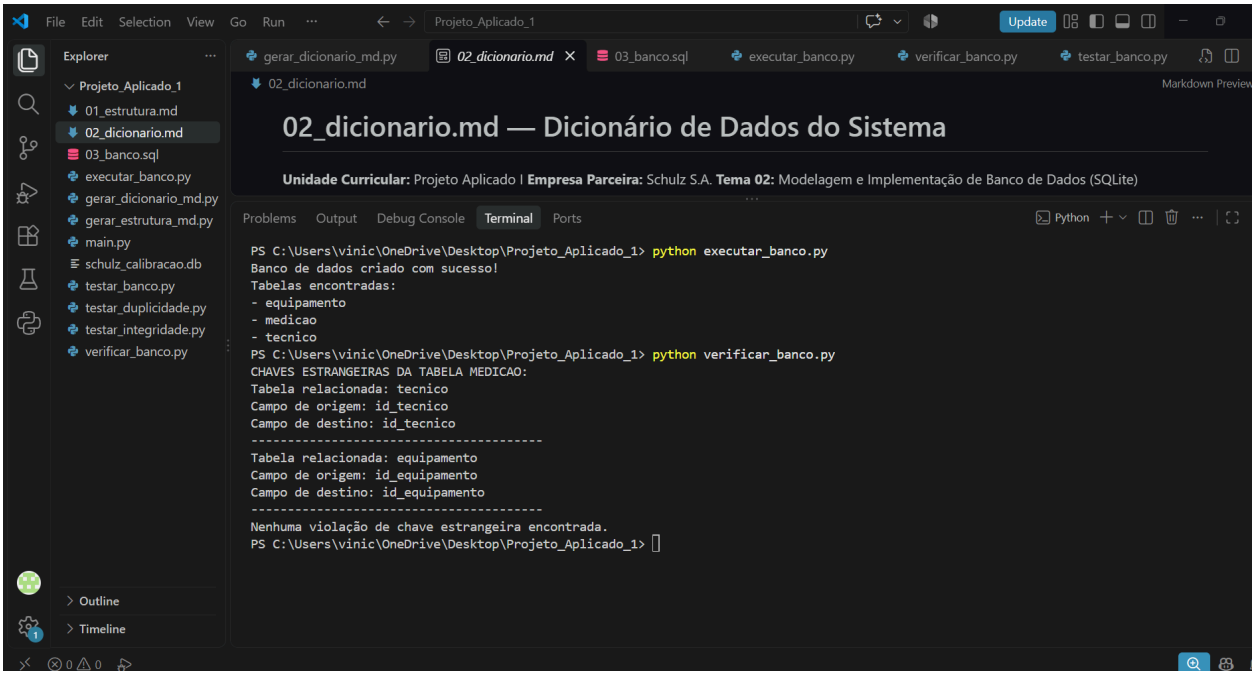
Após a implementação, foram realizados testes para verificar o funcionamento básico do banco e a aplicação das restrições definidas.

5.1. Verificação das chaves estrangeiras

Inicialmente, foi executado o script verificar_banco.py, responsável por consultar os relacionamentos existentes na tabela medicao.

A verificação confirmou a presença das duas chaves estrangeiras, relacionando os campos id_tecnico e id_equipamento às respectivas tabelas de origem.

Também foi realizada uma consulta por meio do comando PRAGMA foreign_key_check, que não identificou violações de integridade referencial nos registros armazenados no momento da verificação.



Fonte: Elaborado pelos autores (2026).

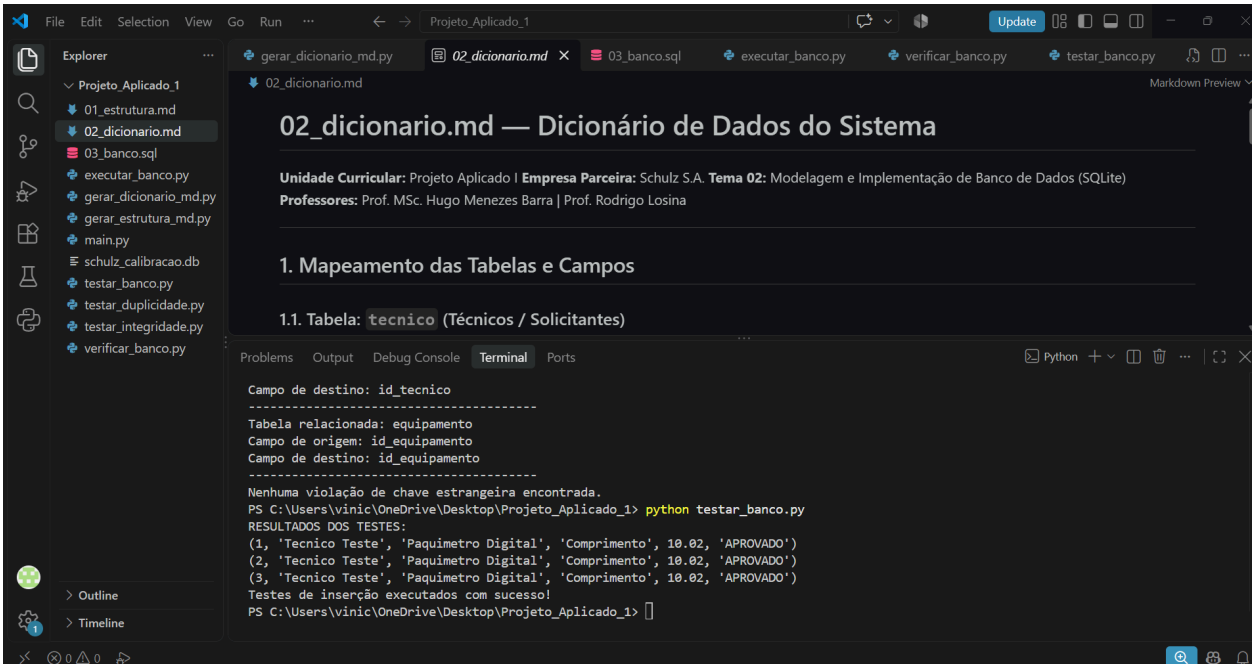
5.2. Testes de inserção e consulta de registros

Para verificar o armazenamento das informações, foi desenvolvido o script testar_banco.py, utilizando dados fictícios de um técnico e um equipamento.

Os registros foram inseridos nas respectivas tabelas e, posteriormente, associados a medições de comprimento. Os valores utilizados tiveram finalidade exclusivamente demonstrativa, sem representar resultados reais de calibração da Schulz S.A.

Após a inserção, foi realizada uma consulta com o comando SELECT, utilizando operações JOIN para relacionar os dados das três tabelas.

O resultado apresentou corretamente o técnico responsável, o equipamento, o parâmetro medido, o valor obtido e o status informado. A consulta demonstrou que os registros estavam armazenados e que os relacionamentos permitiam recuperar informações de tabelas diferentes.



Fonte: Elaborado pelos autores (2026).

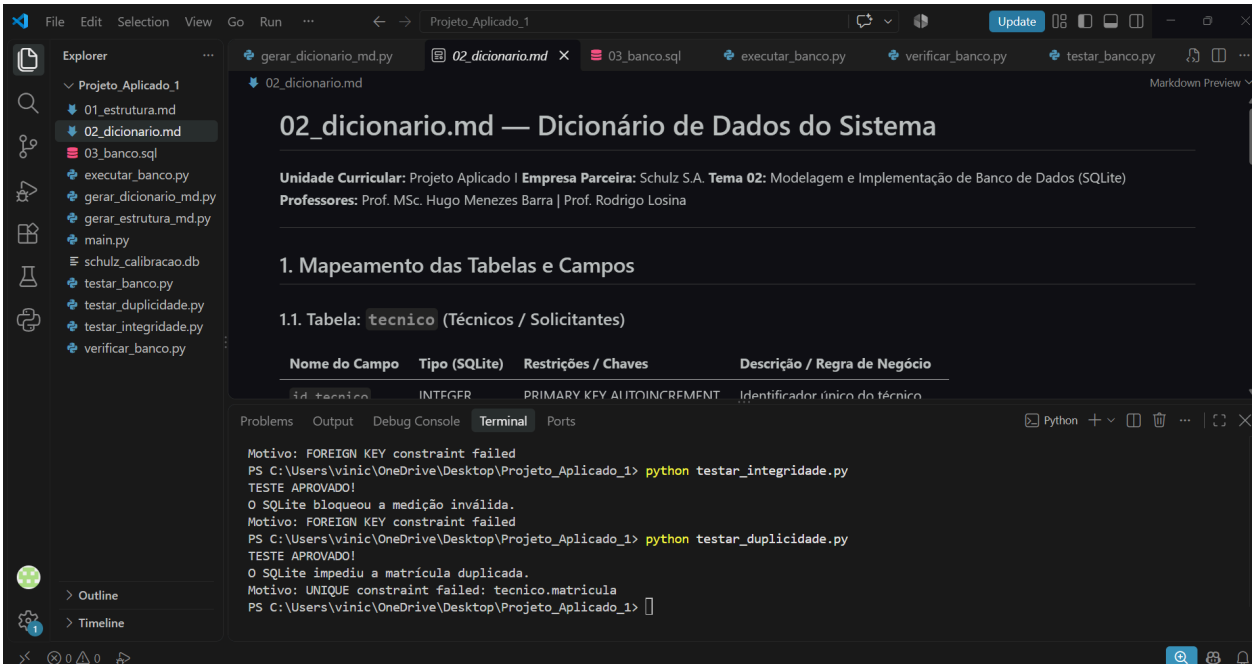
5.3. Testes das restrições de integridade

Também foram realizados testes específicos para verificar se o banco impediria operações incompatíveis com as regras estabelecidas.

No primeiro teste, executado pelo script testar_integridade.py, foi realizada uma tentativa de inserir uma medição associada a um identificador de equipamento inexistente. O SQLite bloqueou a operação e apresentou a mensagem FOREIGN KEY constraint failed, confirmando a aplicação da restrição de chave estrangeira.

No segundo teste, executado pelo script testar_duplicidade.py, foi realizada uma tentativa de cadastrar outro técnico utilizando uma matrícula já existente. O SQLite rejeitou a operação e apresentou a mensagem UNIQUE constraint failed: tecnico.matricula, demonstrando que a restrição de unicidade estava funcionando.

Os resultados comprovaram, nos cenários testados, que o banco consegue impedir determinados registros inválidos e preservar os relacionamentos entre as tabelas.



Fonte: Elaborado pelos autores (2026).

6. Limitações da Implementação

A estrutura desenvolvida corresponde a uma implementação inicial e simplificada do banco de dados, conforme o escopo estabelecido para esta entrega.

Embora permita registrar medições e relacioná-las aos equipamentos e técnicos, o modelo ainda não contempla todas as informações previstas no levantamento de requisitos realizado na Atividade 1.

Entre as limitações identificadas está a ausência de uma estrutura específica para agrupar as medições de uma mesma calibração e identificar suas repetições. No levantamento anterior, foram identificadas características avaliadas duas vezes, cuja correspondência precisa ser preservada no sistema completo.

Também não foram implementadas as funcionalidades de importação automática das planilhas, cálculo da incerteza de medição, avaliação metrológica automática ou geração dos certificados de calibração.

O campo status_resultado armazena valores previamente informados, limitados às opções APROVADO e REPROVADO. A classificação automática dos resultados, com base em critérios metrológicos formalmente aprovados, permanece como possibilidade de desenvolvimento futuro.

Portanto, a implementação atual atende à organização inicial das informações, sem representar a conclusão do sistema de automação dos certificados.

7. Arquivos do Projeto e Repositório

Os principais arquivos desenvolvidos para esta entrega são:

- 01_estrutura.md: documentação da organização das entidades e justificativa para redução da duplicação de dados.
- 02_dicionario.md: descrição dos 20 campos utilizados nas três tabelas.
- 03_banco.sql: comandos SQL responsáveis pela criação das tabelas e definição das restrições.
- executar_banco.py: script Python utilizado para criar o banco de dados SQLite.
- schulz_calibracao.db: arquivo que armazena o banco de dados implementado.

Também foram desenvolvidos scripts auxiliares para geração da documentação e realização dos testes de inserção e integridade.

Repositório GitHub: https://github.com/viniciuscon2729-cyber/Projeto_Aplicado_1_Schulz